

Name: - Shubham Das

S. A. P I. D: - 590033500

Date: - 07/09/2026

Programme: - B.TECH (CSE)

Assignment: - Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases. Write a menu-driven C programme using switch-case to input and perform one of the following operations based on the user's choice:

- 1. Check whether all its digits are unique.**
- 2. Find the frequency of each digit from '0' to '9'.**

Part 1: - Thorough understanding of the problem statement

Input: - 'num' (Positive integer to be processed, Variable type: 'int', Unit: unitless integer)

'choice' (Menu option selection, Variable type: 'int', Unit: unitless integer)

Output: - Option 1: - Uniqueness status messages ("Unique" or "Not Unique",
Variable type: text output)

Option 2: Individual frequencies for digits '0' through '9' ('freq_0' to 'Freq_9', Variable type: 'int' array of size 10)

Formula: - Digit Extraction: 'digit = temp % 10';

Digit Truncation: 'temp = temp / 10';

Frequency Counter: 'freq[digit]++';

Uniqueness Logic: If 'freq[digit] > 1', set 'is_unique = 0' and break Early.

Part 2: - Writing and evaluating the algorithm

Step 1: - Start

Step 2: - Declare integer variables 'num', 'temp', 'choice', 'digit', 'i', 'is_unique' (initialize to 1), and array 'freq[10]' (initialize all 10 elements to 0).

Step 3: - Prompt the user to enter a positive integer and store it in 'num'.

Step 4: - Validate 'num'. If 'num <= 0', display an error message and terminate the programme.

Step 5: - Display the menu options (1: Check if digits are unique, 2: Find frequency of each digit).

Step 6: - Read the user's 'choice'.

Step 7: - Initiate a 'switch-case' block based on 'choice'.

Step 8 (Case 1): - Set 'temp = num'. Using a 'while (temp > 0)' loop: extract 'digit = temp % 10', increment 'freq[digit]++'. If 'freq[digit] > 1', set 'is_unique = 0' and break loop. Update 'temp = temp / 10'. If

"is _unique == 1", print "Unique", else print "Not Unique". Break.

Step 9 (Case 2): - Set 'temp = num'. Using a 'while (temp > 0)', loop: extract 'digit = temp % 10', increment 'freq[digit]++', update 'temp = temp / 10'. Loop index 'i' from 0 to 9 and print 'freq_i = freq[i]'. Break.

Step 10 (Default): - Print an 'Invalid choice' error message.

Step 11: Stop

Part 3: - Writing and compiling the C programme

kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

1 2 3 4

kali@kali: ~

Session Actions Edit View Help

GNU nano 9.0 unique_freq.c

```
#include <stdio.h>

int main() {
    int num, temp, choice, digit, i;
    int freq[10] = {0};
    int is_unique = 1;

    // Input and Validation
    printf("Enter a positive integer: ");
    if (scanf("%d", &num) != 1 || num <= 0) {
        printf("Error: Please enter a strictly positive integer greater than zero.\n");
        return 1;
    }

    // Menu Display
    printf("\n--- Digit Analysis Menu ---\n");
    printf("1. Check whether all digits are unique\n");
    printf("2. Find frequency of each digit (0 to 9)\n");
    printf("Enter your choice (1 or 2): ");
    scanf("%d", &choice);

    printf("\n--- Result ---\n");
    switch (choice) {
        case 1:
            temp = num;
            while (temp > 0) {
                digit = temp % 10;
                freq[digit]++;
                if (freq[digit] > 1) {
                    is_unique = 0;
                    break; // Early exit upon finding any duplicate digit
                }
                temp /= 10;
            }

            printf("Number: %d\n", num);
            if (is_unique) {
                printf("Output: Unique\n");
            } else {
                printf("Output: Not Unique\n");
            }
            break;

        case 2:
            temp = num;
            while (temp > 0) {
                digit = temp % 10;
                freq[digit]++;
            }
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

1 2 3 4

Session Actions Edit View Help

GNU nano 9.0

```
scanf("%d", &choice);

printf("\n--- Result ---\n");
switch (choice) {
    case 1:
        temp = num;
        while (temp > 0) {
            digit = temp % 10;
            freq[digit]++;
            if (freq[digit] > 1) {
                is_unique = 0;
                break; // Early exit upon finding any duplicate digit
            }
            temp /= 10;
        }

        printf("Number: %d\n", num);
        if (is_unique) {
            printf("Output: Unique\n");
        } else {
            printf("Output: Not Unique\n");
        }
        break;

    case 2:
        temp = num;
        while (temp > 0) {
            digit = temp % 10;
            freq[digit]++;
            temp /= 10;
        }

        printf("Number: %d\n", num);
        for (i = 0; i < 10; i++) {
            printf("freq_%d = %d", i, freq[i]);
            if (i < 9) {
                printf(", ");
            }
        }
        printf("\n");
        break;

    default:
        printf("Error: Invalid choice. Please select 1 or 2.\n");
        return 1;
}

return 0;
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

Step 1: - Open the terminal

Step 2: - Create the source code

Step 3: - Write and Save Code

Step 4: - Compile the programme

Step 5: - Run the Binary Executable programme



Session Actions Edit View Help

```
(kali㉿kali)-[~]
```

```
$ nano unique_freq.c
```

```
(kali㉿kali)-[~]
```

```
$ gcc unique_freq.c -o unique_freq
```

```
(kali㉿kali)-[~]
```

```
$ ./unique_freq
```

```
Enter a positive integer: 12345
```

```
--- Digit Analysis Menu ---
```

```
1. Check whether all digits are unique
```

```
2. Find frequency of each digit (0 to 9)
```

```
Enter your choice (1 or 2): 1
```

```
--- Result ---
```

```
Number: 12345
```

```
Output: Unique
```

```
(kali㉿kali)-[~]
```

```
$ ./unique_freq
```

```
Enter a positive integer: 12234
```

```
--- Digit Analysis Menu ---
```

```
1. Check whether all digits are unique
```

```
2. Find frequency of each digit (0 to 9)
```

```
Enter your choice (1 or 2): 1
```

```
--- Result ---
```

```
Number: 12234
```

```
Output: Not Unique
```

```
(kali㉿kali)-[~]
```

```
$ ./unique_freq
```

```
Enter a positive integer: 112333
```

```
--- Digit Analysis Menu ---
```

```
1. Check whether all digits are unique
```

```
2. Find frequency of each digit (0 to 9)
```

```
Enter your choice (1 or 2): 2
```

```
--- Result ---
```

```
Number: 112333
```

```
freq_0 = 0, freq_1 = 2, freq_2 = 1, freq_3 = 3, freq_4 = 0, freq_5 =  
0, freq_6 = 0, freq_7 = 0, freq_8 = 0, freq_9 = 0
```

```
(kali㉿kali)-[~]
```

```
$ ./unique_freq
```

```
Enter a positive integer: -12
```



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

--- Result ---
Number: 12345
Output: Unique

(kali@kali)-[~]
$ ./unique_freq
Enter a positive integer: 12234

--- Digit Analysis Menu ---
1. Check whether all digits are unique
2. Find frequency of each digit (0 to 9)
Enter your choice (1 or 2): 1

--- Result ---
Number: 12234
Output: Not Unique

(kali@kali)-[~]
$ ./unique_freq
Enter a positive integer: 112333

--- Digit Analysis Menu ---
1. Check whether all digits are unique
2. Find frequency of each digit (0 to 9)
Enter your choice (1 or 2): 2

--- Result ---
Number: 112333
freq_0 = 0, freq_1 = 2, freq_2 = 1, freq_3 = 3, freq_4 = 0, freq_5 = 0,
freq_6 = 0, freq_7 = 0, freq_8 = 0, freq_9 = 0

(kali@kali)-[~]
$ ./unique_freq
Enter a positive integer: -12
Error: Please enter a strictly positive integer greater than zero.

(kali@kali)-[~]
$ ./unique_freq
Enter a positive integer: 555

--- Digit Analysis Menu ---
1. Check whether all digits are unique
2. Find frequency of each digit (0 to 9)
Enter your choice (1 or 2): 3

--- Result ---
Error: Invalid choice. Please select 1 or 2.
```

Part 5: - Standard test cable

TEST CASE	INPUT (num, choice)	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT (PASS/FAIL)
1	num = 12345 choice = 1	Output: Unique	Output: Unique	PASS
2	num = 12234 choice = 1	Output: Not Unique	Output: Not Unique	PASS
3	num = 112333 choice = 2	freq_0 = 0 freq_1 = 2 freq_2 = 1 freq_3 = 3 freq_4 = 0 freq_5 = 0 freq_6 = 0 freq_7 = 0 freq_8 = 0 freq_9 = 0	freq_0 = 0 freq_1 = 2 freq_2 = 1 freq_3 = 3 freq_4 = 0 freq_5 = 0 freq_6 = 0 freq_7 = 0 freq_8 = 0 freq_9 = 0	PASS
4	num = -12 choice = 1	Error: strictly positive integer	Error: strictly positive integer	PASS
5	num = 555 choice = 3	Error: Invalid choice	Error: Invalid choice	PASS

Part 6:- Evaluation

Modulo operations safely map all extracted digits strictly within the \$0-9\$ range, eliminating buffer overflow risks.